02

Injeção de Dependências e Service Container

Injeção de Dependências e Service Container

Evolução do projeto dia1 com exemplos de injeção de dependências no Laravel.

Roteiro da aula

- Injeção de dependências
- Interface como contrato
- Deep dive no Container
- Binding simples
- Singleton
- Contextual Binding
- Service Providers
- Aplicação modular

Injeção de dependências

Definição
Injeção de dependências é receber de fora aquilo que a classe precisa para trabalhar.

Exemplo

Pontos

- A classe declara o que precisa.
- O Laravel entrega a dependência.

Exemplo de código

```
class ProductService
{
    public function __construct(
        private ProductRepositoryInterface $repository,
    ) {}
}
```

Interface como contrato

Definição
Interface é um contrato. Ela diz quais métodos uma implementação precisa ter.

Exemplo

Implementação no projeto

Exemplo de código

```
interface ProductRepositoryInterface
{
    public function all(): array;
}
```

Deep dive no Container

O Service Container resolve objetos para a aplicação.

Ele consegue

- criar ProductService automaticamente;
- injetar ProductRepositoryInterface no construtor;
- associar a interface a ProductRepository;
- compartilhar ProductService quando necessário.

Termos

- bind: cria uma instância a cada resolução.
- singleton: reutiliza a mesma instância.

Binding simples

Exemplo

Leitura

"Quando alguém pedir ProductRepositoryInterface, entregue ProductRepository."

Exemplo de código

```
$this->app->bind(
    ProductRepositoryInterface::class,
    ProductRepository::class,
);
```

Singleton

Exemplo

No projeto dia1

- ProductService sem estado mutável;
- repositório injetado pelo Container;
- uma instância compartilhada durante a execução.

Cuidado

Singleton mal usado pode manter estado entre chamadas e gerar bugs difíceis.

Exemplo de código

```
$this->app->singleton(
    ProductService::class,
);
```

Service Providers

Definição
Service Provider é o lugar onde registramos serviços, bindings e configurações da aplicação.

Exemplo

Exemplo de código

```
class AppServiceProvider extends ServiceProvider
{
    public function register(): void
    {
        $this->app->bind(
            ProductRepositoryInterface::class,
            ProductRepository::class,
        );
    }
}
```

Aplicação modular

Organização sugerida

Pontos

- O contrato fica no domínio Product.
- A implementação Eloquent fica em Repositories.
- O AppServiceProvider registra o binding.

Exemplo de código

```
app/
  Domains/
    Product/
      Interfaces/
        ProductRepositoryInterface.php
      Repositories/
        Eloquent/
      ProductRepository.php
        Providers/AppServiceProvider.php
        Services/ProductService.php
```